LAB | Automating EBS Snapshot Cleanup with AWS Lambda Extra

Skills you will learn

Cloud Computing: Compute & Networking

Introduction

In this hands-on lab, you will create an AWS Lambda function that automates the cleanup of EBS snapshots that are no longer associated with active EC2 instances. This process helps optimize storage costs and ensures efficient resource management.

Objectives

By completing this lab, you will:

1. Create a Python-based AWS Lambda function.
2. Configure IAM permissions for Lambda to interact with EC2.
3. Automate the identification and deletion of unused EBS snapshots.
4. Test the Lambda function with both active and inactive snapshots.

Prerequisites

- An active AWS account with necessary permissions.
- Basic knowledge of EC2, EBS, IAM roles, and AWS Lambda.
- AWS CLI installed and configured (optional but recommended).

Step 1: Create a Lambda Function

1. **Navigate** to the AWS Management Console and open the **Lambda** service.
2. **Click** on **Create function**.
3. Select **Author from scratch** and configure the function:
   - **Function name**: `cleanup-ebs-snapshots`

Ask AI Tutor

- **Runtime**: `Python 3.x`
- **Execution role**: Create a new role with basic Lambda permissions.

4. **Click** on **Create function**.

Step 2: Add and Deploy the Lambda Function Code

1. **Replace** the default code in the function editor with the following:

Copy Explain

```
import boto3

def lambda_handler(event, context):
    ec2 = boto3.client('ec2')

    # Get all EBS snapshots
    response = ec2.describe_snapshots(OwnerIds=['self'])

    # Get all active EC2 instance IDs
    instances_response = ec2.describe_instances(Filters=[{'Name': 'instance-
state-name', 'Values': ['running']}])
    active_instance_ids = set()

    for reservation in instances_response['Reservations']:
        for instance in reservation['Instances']:
            active_instance_ids.add(instance['InstanceId'])

    # Identify and delete orphaned snapshots
    for snapshot in response['Snapshots']:
        snapshot_id = snapshot['SnapshotId']
        volume_id = snapshot.get('VolumeId')

        if not volume_id:
            ec2.delete_snapshot(SnapshotId=snapshot_id)
            print(f"Deleted EBS snapshot {snapshot_id} (No associated volume).")
        else:
            try:
                volume_response = ec2.describe_volumes(VolumeIds=[volume_id])
                if not volume_response['Volumes'][0]['Attachments']:
                    ec2.delete_snapshot(SnapshotId=snapshot_id)
                    print(f"Deleted EBS snapshot {snapshot_id} (Detached volume).")
            except ec2.exceptions.ClientError as e:
                if e.response['Error']['Code'] == 'InvalidVolume.NotFound':
                    ec2.delete_snapshot(SnapshotId=snapshot_id)
                    print(f"Deleted EBS snapshot {snapshot_id} (Volume not found).")
```

2. **Click** on **Deploy** to save and activate the function.

Step 3: Update Execution Timeout

1. **Go to** the **Configuration** tab in Lambda.

2. **Click** on **Edit** under **General configuration**.
3. **Set Timeout** to `10 seconds`.
4. **Click** on **Save**.

Step 4: Configure IAM Role Permissions

1. **Go to** the AWS IAM service.
2. **Find** the execution role assigned to your Lambda function.
3. **Attach the following permissions** to the role:
   - `ec2:DescribeSnapshots`
   - `ec2:DeleteSnapshot`
   - `ec2:DescribeVolumes`
   - `ec2:DescribeInstances`
4. **Save the role.**

Step 5: Create and Verify an EC2 Snapshot

1. **Launch** a new EC2 instance from the **EC2 Dashboard**.
2. **Navigate** to **Volumes**, select the volume attached to the instance, and click **Create Snapshot**.
3. **Note** the Snapshot ID for later verification.

Step 6: Test the Lambda Function

1. **Return to** the AWS Lambda console.
2. **Click** on **Test** and create a new test event (leave the default template blank).
3. **Run the test**.
4. **Check** the Lambda console output or **CloudWatch Logs** for snapshot deletion messages.

Step 7: Terminate EC2 and Re-Test

1. **Terminate** the EC2 instance from **EC2 Dashboard**.
2. **Re-run** the Lambda function.
3. **Verify** that the snapshot is deleted since it no longer belongs to an active volume.

Expected Results

- **After Step 6**: The snapshot remains since the EC2 instance is active.
- **After Step 7**: The snapshot is deleted because the EC2 instance and its associated volume are no longer active.

Key Learning Points

1. Automating snapshot cleanup reduces storage costs.
2. Configuring IAM permissions correctly ensures security.
3. CloudWatch Logs provide insights into automation behavior and debugging.

Cleanup

To avoid unnecessary charges, delete all created resources:

1. **Delete** the Lambda function from the Lambda console.
2. **Remove** the IAM role and associated permissions.
3. **Ensure** all test snapshots are deleted.

Submission Guidelines

- Submit screenshots showing:
  - Lambda function creation and configuration.
  - IAM role setup.
  - Execution results in the Lambda console.
- Provide a brief summary of any challenges faced and solutions implemented.

By completing this lab, you have successfully automated EBS snapshot cleanup using AWS Lambda, optimizing storage costs and improving AWS resource management. 🚀

LAB | Automating EBS Snapshot Cleanup with AWS Lambda

Extra

URL

Enter the URL here

Submit lab

Day 4 Quiz